

ԿՈՂԻ ԲԱՑԱՏՐՈՒԹՅՈՒՆ

Ինչպես է աշխատում ծրագիրը (քայլ առ քայլ)

Թեմա՝	Կոմպիլյատորի և վիրտուալ մեքենայի մշակումը
Ուսանող՝	Արման Խանգադյան
Դասախոս՝	Խաչատրյան Տ.

ԸՆԴՀԱՆՈՒՐ ԴԱՏԿԵՐ

Այս փաստաթուղթը ցույց է տալիս, թե **ինչպես է աշխատում կոդը`** ֆայլ առ ֆայլ, իրական հատվածներով (snippet): Վերջում կան **կոդի վերաբերյալ հարցեր և պատասխաններ**, որոնք դասախոսը կարող է տալ կոդը դիտելիս:

Ծրագիրը կազմված է փուլերից: Ամեն ֆայլ պատասխանատու է մեկ փուլի համար.

Ֆայլ	Ինչ է անում
<code>include/Lexer.h</code>	տեքստ → token-ներ
<code>include/Parser.h</code>	token-ներ → AST
<code>src/IRGen.cpp</code>	AST → միջանկյալ կոդ (TAC)
<code>src/LogicalCode.cpp</code>	&& ! → ճյուղավորումներ
<code>src/CodeGenIR.cpp</code>	IR → երկուական հրամաններ
<code>src/optimizer.cpp</code>	peephole օպտիմիզացիա
<code>src/linker.cpp</code>	սիմվոլների/relocation-ների լուծում
<code>src/ExecIO.cpp</code>	.exec ֆայլի գրառում/կարդացում
<code>src/vm.cpp</code>	վիրտուալ մեքենա (կատարում)
<code>src/debugger.cpp</code>	վրիպագերծիչ
<code>src/main.cpp</code>	միացնում է բոլոր փուլերը

Կատարման մուտքը `main()` -ն է `src/main.cpp` -ում, որը հերթականությամբ կանչում է բոլոր փուլերը:

1. LEXER — ինչպես է աշխատում

```
include/Lexer.h
```

`tokenize()`-ը պատվում է տեքստի վրա, բաց է թողնում բացատներն ու մեկնաբանությունները, ապա ըստ առաջին սիշի որոշում ինչ token կարդալ.

```
while (pos < src.size()) {
    skipWhitespace();
    // // մեկնաբանություն
    if (src[pos]=='/' && src[pos+1]=='/') { ...բաց թողնել տողը... }
    char c = src[pos];
    if (isdigit(c))         tokens.push_back(readNumber());
    else if (isalpha(c)||c=='_') tokens.push_back(readIdent());
    else                   tokens.push_back(readSymbol());
}
```

Բանալի բառերը տարբերվում են անուններից `readIdent()`-ում. կարդում է ամբողջ բառը, ապա համեմատում ցուցակի հետ.

```
std::string t = ...ամբողջ բառը...;
if (t=="if")    return { TokenType::IF, t, line };
if (t=="while") return { TokenType::WHILE, t, line };
// եթե ոչ մի keyword չհամընկնի՝ սա identifier է
return { TokenType::VARIABLE, t, line };
```

Երկնիշ օպերատորները (`== != <= &&`) ճանաչվում են «peek»-ով. կարդալուց հետո նայում է հաջորդ սիշին.

```
case '=':
    if (peek()=='=') { pos++; return {EQ, "=="}; }
    return {ASSIGN, "="};
```

2. PARSER — ինչպես է աշխատում

`include/Parser.h`

Parser-ը **ռեկուրսիվ իջնող** է: `parseProgram()` -ը կարողում է հայտարարությունները մինչև ֆայլի վերջ.

```
while (!check(END_OF_FILE))  
    prog->children.push_back(parseDeclaration());
```

Օպերատորների առաջնահերթությունը իրականացված է ֆունկցիաների շղթայով՝ ամեն մակարդակ կանչում է ավելի բարձր առաջնահերթության մակարդակը: Այսպես `*`-ը խմբավորվում է `+`-ից առաջ.

```
parseAddSub() {  
    // + -  
    left = parseMulDiv(); // նախ * / %  
    while (PLUS || MINUS) {  
        op = advance();  
        node = BINOP(op);  
        node.children = { left, parseMulDiv() };  
        left = node;  
    }  
    return left;  
}
```

Արդյունքում ստացվում է AST: Օրինակ՝ `2 + 3 * 4`-ի համար `3 * 4`-ը դառնում է ներքին հանգույց, ուստի ճիշտ հաշվարկվում է որպես 14:

Սխալների մշակում. `consume()` -ը, եթե սպասվող token-ը չի գտնում, նետում է սխալ՝ տողի համարով. `"Parse error at line N: ..."`:

3. IRGen — AST-ից միջանկյալ կոդ

src/IRGen.cpp

`emitExpr()`-ը ռեկուրսիվ իջնում է AST-ով և արտադրում եռհասցեանի հրամաններ՝ ժամանակավոր `t0, t1` փոփոխականներով.

```
case BINOP: {
    std::string l = emitExpr(child0); // ձախ կողմ
    std::string r = emitExpr(child1); // աջ կողմ
    std::string t = fresh();           // նոր t0, t1 ...
    emit({ IROp::ADD, t, l, r });      // t = l + r
    return t;
}
```

Վերագրումը (`c = a + b`) մշակվում է այսպես՝ նախ հաշվում աջ կողմը, ապա պատճենում ձախ կողմ.

```
case ASSIGN_EXPR: {
    std::string rhs = emitExpr(child1);
    std::string dst = emitExpr(child0);
    emit({ IROp::MOV_REG, dst, rhs, "" });
    return dst;
}
```

Ուղղված սխալ. Սկզբում `ASSIGN_EXPR`-ի մշակումը բացակայում էր, ուստի վերագրումները կորչում էին, և ծրագիրը սխալ վերադարձնում էր 0: Այս `case`-ը ավելացնելուց հետո ծրագիրը ճիշտ վերադարձնում է 15:

4. CodeGenIR և Optimizer

src/CodeGenIR.cpp

Կողի գեներատորը IR-ից արտադրում է ֆիքսված 4-բայթանոց հրամաններ: IR-ի անունները ստանում են ռեգիստրների համարներ `regOf()`-ով.

```
case IROp::ADD: {
    int d = regOf(dst), a = regOf(src1), b = regOf(src2);
    emit({ OpCode::ADD, d, a, b }); // երկուական հրաման
}
```

Ֆունկցիայի կանչը գրանցում է **relocation** (հասցեն դեռ հայտնի չէ).

```
case IROp::CALL:
    emit({ OpCode::CALL, d, 0, 0 });
    prog_.relocs.push_back({ ci, src1 }); // թիրախ՝ անուկով
```

src/optimizer.cpp

Peephole օպտիմիզացիան հեռացնում է «մեռած» գրառումը՝ երբ երկու `MOV_CONST` իրար հետևից գրում են նույն ռեգիստրը.

```
if (a.op==MOV_CONST && b.op==MOV_CONST && a.rd==b.rd)
    code.erase(code.begin()+i); // առաջինը ավելորդ է
```

5. LINKER — հասցեների լուծում

src/linker.cpp

Linker-ը անցնում է relocation-ների վրայով, ամեն թիրախ որոնում է symbol table-ում ըստ անվան և իրական հասցեն գրում կանչի հրամանի մեջ.

```
// անուն → հասցե աղյուսակ
for (s : prog.symbols) symbolAddr[s.name] = s.address;

// ամեն կանչ լրացնել իրական հասցեով
for (r : prog.relocs) {
    auto it = symbolAddr.find(r.targetName);
    if (it == symbolAddr.end()) { unresolved_++; continue; }
    prog.code[r.instrIndex].rs1 = it->second;
}
return unresolved_ == 0;
```

Եթե կանչվող ֆունկցիան չի գտնվում, այն հաշվվում է որպես **unresolved reference** և հաղորդվում է սխալ: Կապակցումը առանձին փուլ է կոդի գեներացիայից:

6. VIRTUAL MACHINE — կատարում

src/vm.cpp

Մեքենան աշխատում է **fetch** → **decode** → **execute** ցիկլով.

```
void VM::runOne() {
    Instruction instr = fetch();    // 1. կարդալ pc-ից
    pc++;
    decode(instr);                 // 2. վերծանել
    execute();                     // 3. կատարել
}
void VM::run() {
    while (running && pc < code.size()) runOne();
}
```

`execute()` -ը մեծ `switch` է ըստ օպ-ի. օրինակ.

```
case OpCode::ADD:
    regs[rd] = regs[rs1] + regs[rs2]; break;
case OpCode::BR_EQ:
    if (regs[rd]==0) pc = rs2; break; // պայմանական անցում
case OpCode::CALL:
    callStack.push_back(pc); ... pc = rs1; break;
case OpCode::HALT:
    running = false; break;
```

Ունի 32 ռեգիստր, `pc`, `sp`, `bp`, դրոշներ, և 1 ՄԲ հիշողություն
(Code/Data/Heap/Stack):

7. DEBUG BUILD — trace

src/vm.cpp (DEBUG_BUILD)

Debug ռեժիմում ամեն հրաման տպվում է կատարումից առաջ՝ `#ifdef DEBUG_BUILD` պայմանով.

```
#ifdef DEBUG_BUILD
    fprintf(stderr, "[trace] pc=%-4llu %-11s rd=%u rs1=%u rs2=%u\n",
                pc, opCodeNames[instr.op], instr.rd, instr.rs1, instr.rs2);
#endif
```

Արդյունքում երևում է fetch-decode-execute ցիկլը.

```
[trace] pc=0  PUSH_BP      rd=0 rs1=0 rs2=0
[trace] pc=3  ADD          rd=2 rs1=1 rs2=0
...
Result R0 = 15 (expected 15)
```

Հավաքում՝ `make debug`. Release՝ `make` :

ԿՈՂԻ ՀԱՐՑԵՐ ԵՎ ՊԱՏԱՍԽԱՆՆԵՐ (1)

Հարցեր, որ դասախոսը կարող է տալ կողք դիտելիս:

Հ1. Որտեղի՞ց է սկսվում ծրագրի կատարումը:

`main()`-ից՝ `src/main.cpp`-ում, որը հերթականությամբ կանչում է `Lexer`, `Parser`, `IRGen`, `LogicalCode`, `CodeGenIR`, `Optimizer`, `Linker`, `ExecIO`, `VM`:

Հ2. Ինչպե՞ս է `lexer`-ը `keyword`-ը տարբերում `identifier`-ից:

`readIdent()`-ը կարդում է ամբողջ բառը, ապա համեմատում `keyword`-ների ցուցակի հետ: Եթե համընկնում է՝ `keyword token`, հակառակ դեպքում՝ `VARIABLE`:

Հ3. Ինչպե՞ս է ճանաչվում `==`-ը `=`-ից:

`readSymbol()`-ում, `=` կարդալուց հետո «peek»-ով նայում է հաջորդ սիշին. եթե `=` է՝ `EQ (==)`, այլապես՝ `ASSIGN`:

Հ4. Ինչպե՞ս է `parser`-ը ապահովում, որ `*`-ը նախ է `+`-ից:

Առաջնահերթության շղթայով. `parseAddSub()`-ը օպերանդների համար կանչում է `parseMulDiv()`, ուստի բազմապատկումը խմբավորվում է ավելի շուտ:

Հ5. Ի՞նչ է `fresh()`-ը `IRGen`-ում:

Վերադարձնում է նոր ժամանակավոր փոփոխական (`t0`, `t1`, ...) միջանկյալ արդյունքները պահելու համար:

Հ6. Ինչպե՞ս է մշակվում `c = a + b` վերագրումը:

`ASSIGN_EXPR` case-ում՝ նախ հաշվվում է աջ կողմը (`a+b`), ապա `MOV_REG`-ով արդյունքը պատճենվում `c`-ի մեջ:

Հ7. Ինչու՞ է ծրագիրը վերադարձնում 15:

`a=5`, `b=10`, `a<b` ճշմարիտ է, ուստի `c=a+b=15`, որը `return`-վում է ռեգիստր 0-ում (`R0`):

ԿՈՂԻ ՀԱՐՑԵՐ ԵՎ ՊԱՏԱՍԽԱՆՆԵՐ (2)

Հ8. Ի՞նչ է անում `regOf()` -ը CodeGenIR-ում:

IR-ի անունը (`t0`, `a`) կապում է ռեգիստրի համարի հետ: Եթե անունը նոր է՝ տալիս է հաջորդ ազատ ռեգիստրը:

Հ9. Ի՞նչ է `relocation`-ը կոդում:

Գրառում է `{instrIndex, targetName}` ֆունկցիայի կանչի համար, քանի որ հասցեն դեռ հայտնի չէ: Հետո linker-ը այն լրացնում է իրական հասցեով:

Հ10. Ինչպե՞ս է linker-ը գտնում ֆունկցիայի հասցեն:

Symbol table-ից որոնում է ըստ անվան (`symbolAddr[name]`) և գրում `code[r.instrIndex].rs1` -ի մեջ:

Հ11. Ինչպե՞ս է `realiz`-վում `if` -ը մեքենայի մակարդակում:

Կա `CMP` հրաման, որը դնում է արդյունք, ապա `BR_EQ` պայմանական անցում, որը փոխում է `pc` -ն, եթե պայմանը չի բավարարվում:

Հ12. Ինչպե՞ս է աշխատում ֆունկցիայի կանչը VM-ում:

`CALL` -ը պահում է վերադարձի հասցեն և `bp` -ն `callStack` -ում, դնում նոր frame և անցնում ֆունկցիային: `RET` -ը վերականգնում է դրանք և արդյունքը դնում R0-ում:

Հ13. Ինչպե՞ս է կանգ առնում կատարումը:

`HALT` հրամանի դեպքում `running=false`, և `run()` ցիկլը ավարտվում է:

Հ14. Ի՞նչ է անում `#ifdef DEBUG_BUILD` -ը:

Միայն Debug build-ում ակտիվացնում է հրահանգների trace-ը: Release-ում այդ կոդը ընդհանրապես չի կոմպիլյացվում, ուստի արագությունը չի տուժում:

Հ15. Ինչու՞ է կոդը բաժանված շատ ֆայլերի:

Ամեն ֆայլ ունի մեկ պարտականություն (single responsibility). հեշտ է կարդալ, փորձարկել և փոխել: Նաև փուլերը կարող են վերաօգտագործվել: